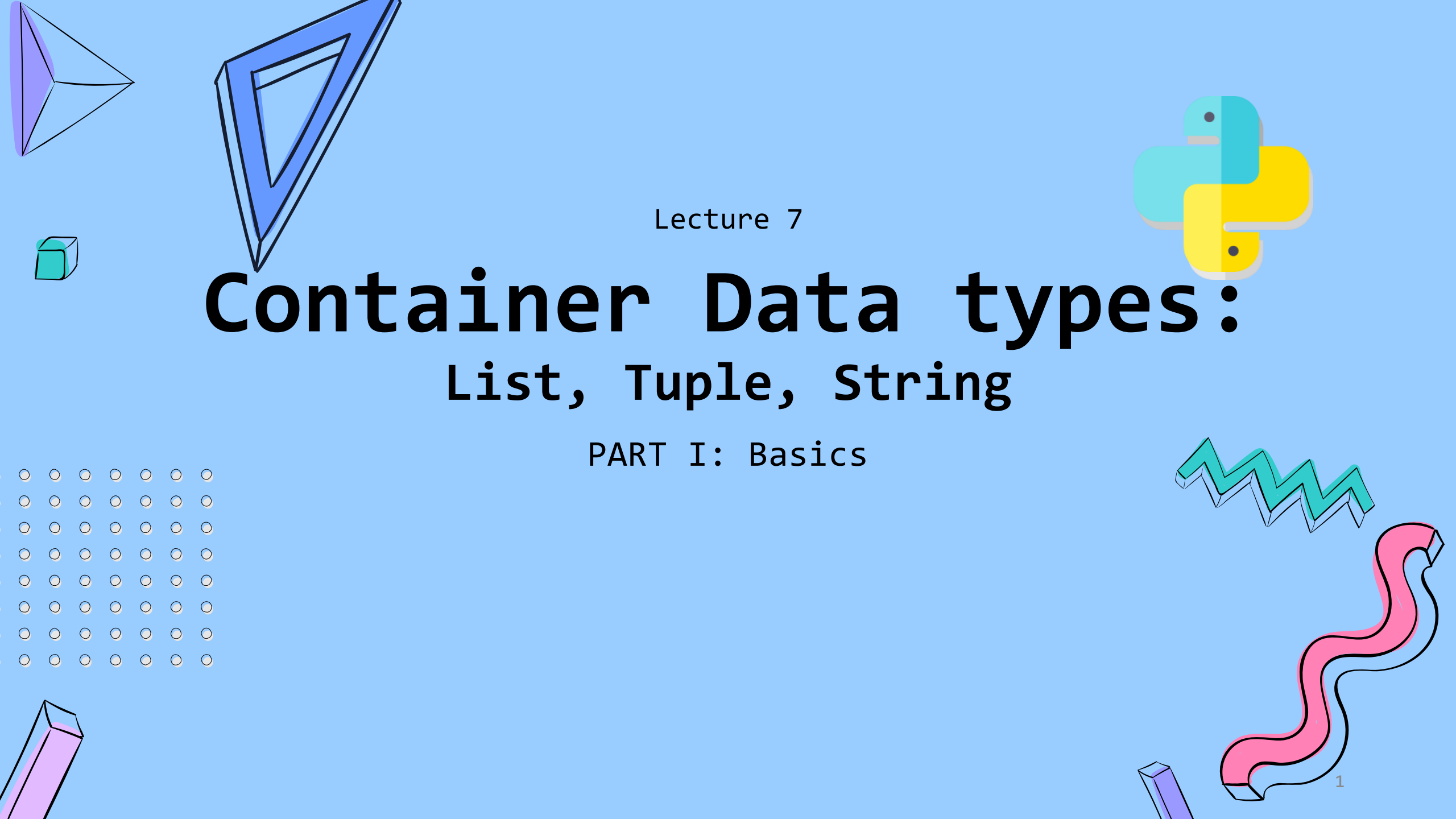


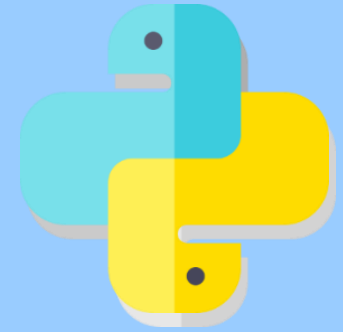


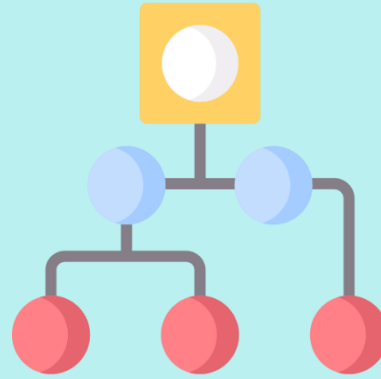
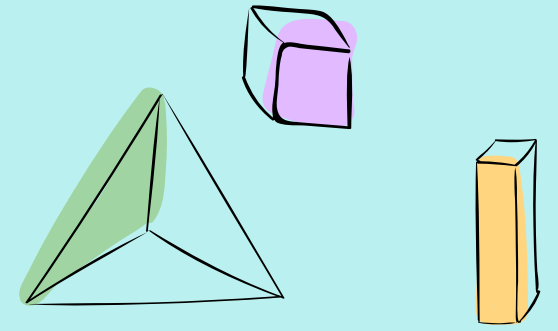
Lecture 7

# Container Data types:

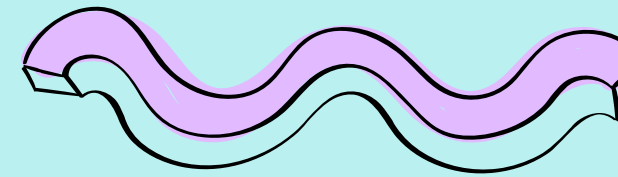
## List, Tuple, String

PART I: Basics





# 01 Classification of Data types





# [1] Classification of data types

■ Python's data types are classified by following Criteria .

- Number of Elements
  - Single / multiple(container)
- Access to elements
  - direct, sequence, mapping, set
- Changeability: Possibility of contents modification
  - mutable, immutable

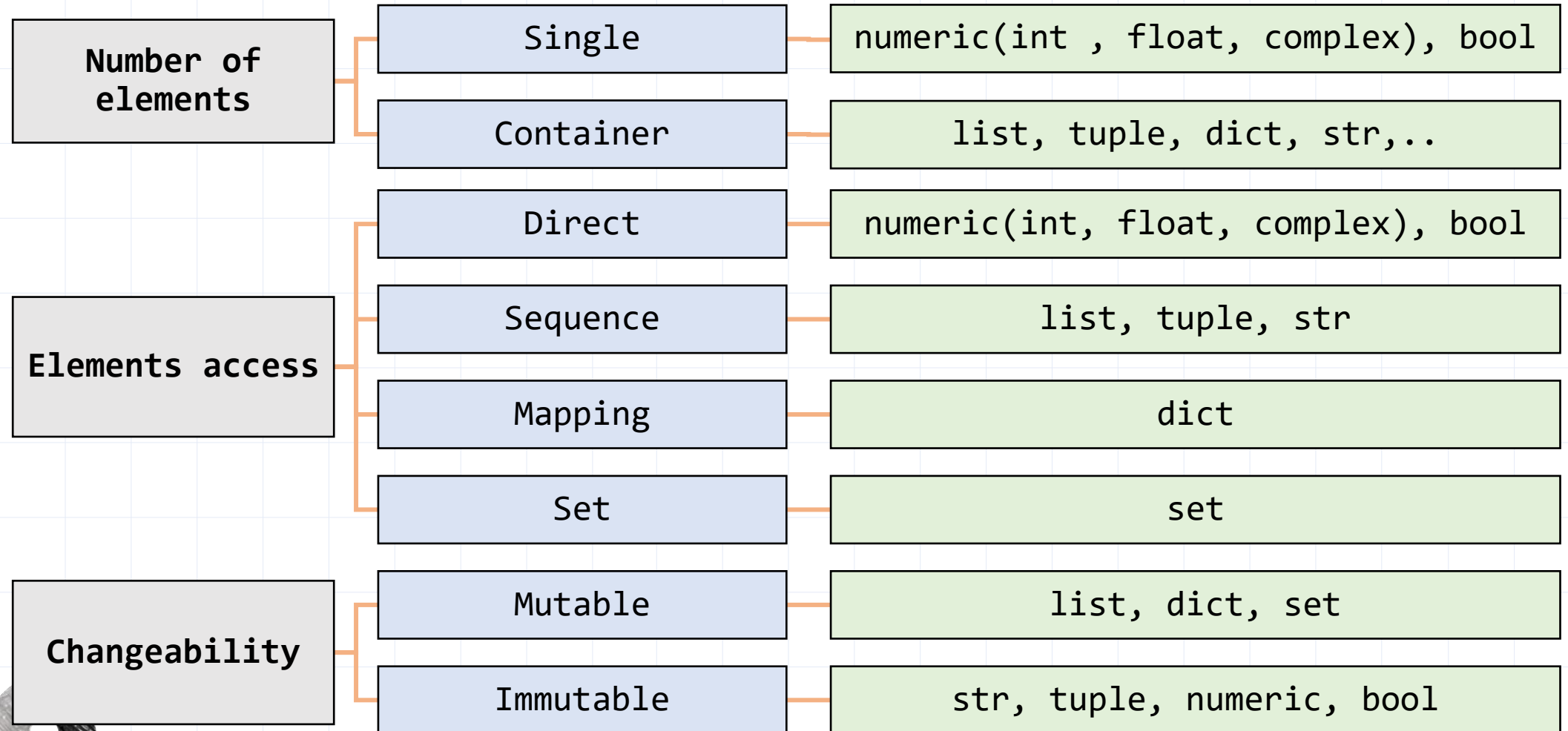
■ Why classification of data types is important

- If they are in the same category, they are similar in how they are used and the methods they provide.

# [1] Classification of data types



## ■ classification chart



## [2] Classification by number of elements



### ■ Classification by number of elements: single vs. container

#### □ single data types:

- data types that can contain only one element

- numeric(numeric typedata):

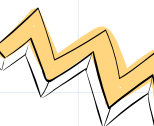
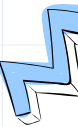
- integers(int), real numbers(float), complex numbers(complex)

- Logic:

- bool: True/False

- example

```
>>> a = 10 ; b = 78.331 ; c = 3.0 - 1.7j
```





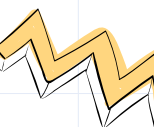
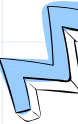
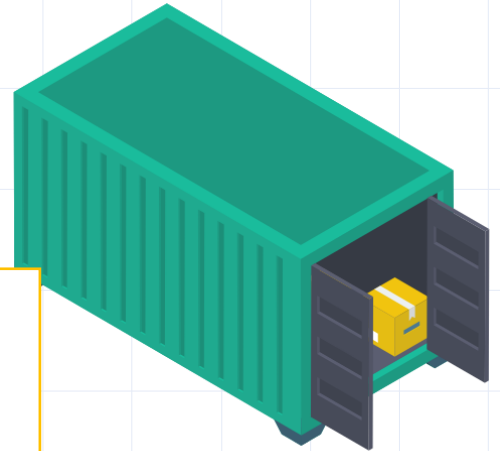
## [2] Classification by number of elements

### □ Container data types :

- data types that can contain multiple elements of arbitrary data types
- List, tuple, dictionary, set, etc.

#### ○ Examples

```
>>> aList = [1, 2, 3, 3.1]           # list
>>> aTuple = (x, y, z)               # tuple
>>> aDic = {1:'one', 2:'two', 3:'three'} # dictionary
>>> aSet = {1, 2, 3, 8}              # set
```



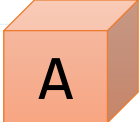
## [3] Classification by element access



### ■ Classification by the methods of accessing elements

- Depending on how the elements are called and written
- *direct*, *sequence*, *mapping*, and *set* access.

#### □ Direct access:

 = 124

- data type that can be accessed directly with just the variable name
- *numeric* and *logic*(boolean) data types belong here.

```
>>> a=124; comp=5+3j; is_okey = True
>>> b=a; q=2*comp; state = is_okey
```

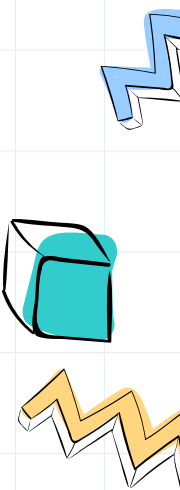
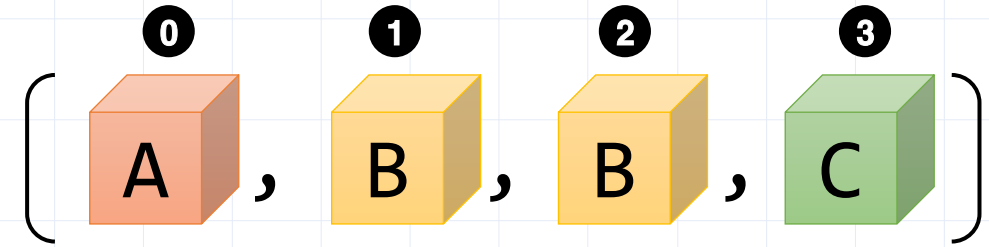
# [3] Classification by element access



## □ Sequence access:

- Square brackets `[]` and index numbers
- Types: *list*, *tuple*, *string*
- Format: *object\_name*[index\_number]
- The order of the elements is important
- Elements can be duplicated
- Example

```
>>> a=[1,2,3,2]           # list
>>> b=a[1]                # 2
>>> coord = (x, y, z)     # tuple
>>> x_coord = coord[0]    # x
```



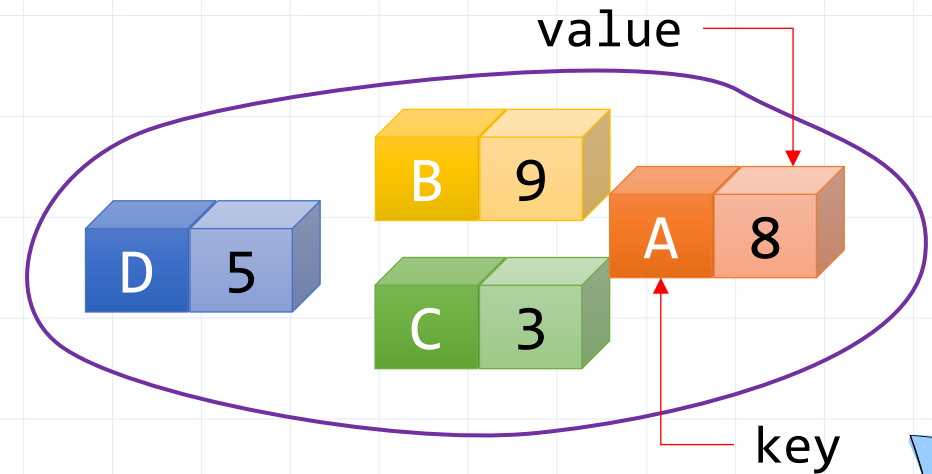
# [3] Classification by element access



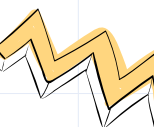
## □ Mapping access:

- A method of accessing elements (values) using keys. Access the value with square brackets `[]` and Key.
- Types: *Dictionary*
- Format : *object\_name*[key]
- Repetition is possible, no order

```
>>> material={'egg':3500, 'ham':7800}  
>>> cost=5*material['egg']
```



A value can be any data type, but a **key must be unique** and an immutable data type.

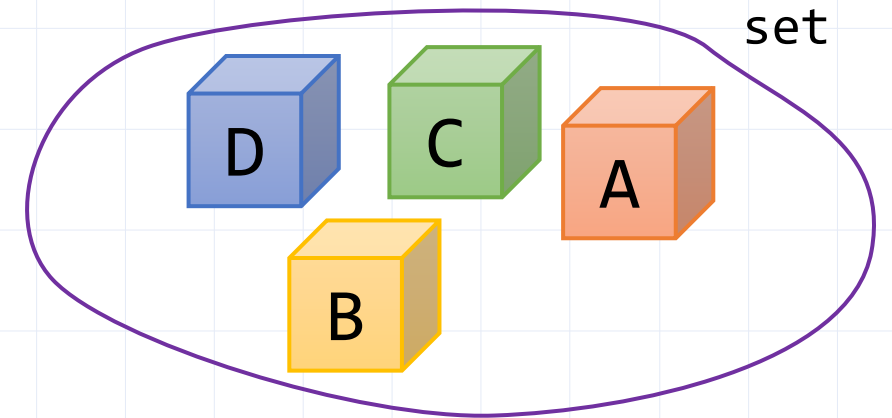
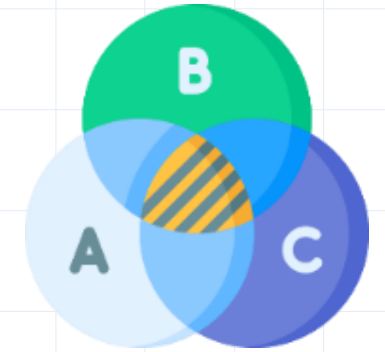


## [3] Classification by element access

### □ Set access

- Individual elements cannot be accessed because there is no index or key.
- Types: set
- Features : no repetition, no order

```
>>> a={1,2,3,4};  
>>> b={1,2,9}  
>>> a.union(b) # union  
{1, 2, 3, 4, 9}
```



**Note:** Access to individual elements is not possible .

## [3] Classification by element modification

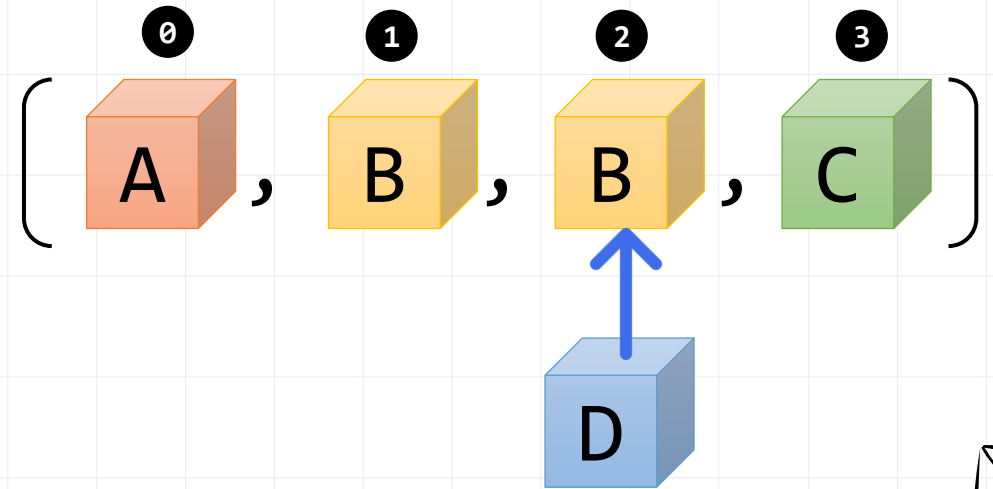


### ■ Classification by Mutability: mutable/immutable

#### □ Mutable:

- A data type is possible to modify the content or number of element.
- Types: *list*, *dictionary*.

```
>>> a=[1,2,3]; a[0]=5  
>>>a  
[5,2,3]  
>>> b={'a':1,'b':2}, b['a']=0  
{'a': 0, 'b': 2}
```

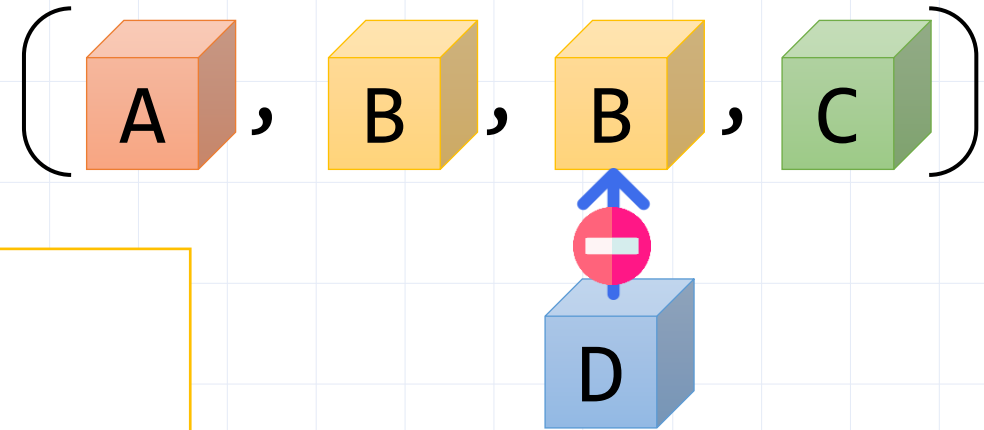


## [3] Classification by element modification



### □ Immutable:

- The contents and number of elements cannot be modified.
- Types: *numeric, string, tuple*
- However, new creation is possible.



Example of Tuple

```
>>> a=(1,2,3); a[0]=5
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    a=(1,2,3); a[0]=5
TypeError : 'tuple' object does not support item assignment
```

Example of string

```
>>> s='abcd '; s[0]='A'
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    s='abcd '; s[0]='A'
TypeError : 'str' object does not support item assignment
```

## [3] Classification by element modification



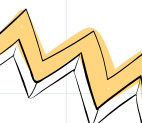
- Example of Numerical data type

```
>>>a=100  
>>>id(a)  
1938782048  
  
>>> a=5  
>>>id(a)  
1938779008
```



***id(object)*** :

This is a built-in function that returns the unique identification number of an object.



# [3] Classification by element modification



## ■ The need for immutable data type??

### ☐ Mutable

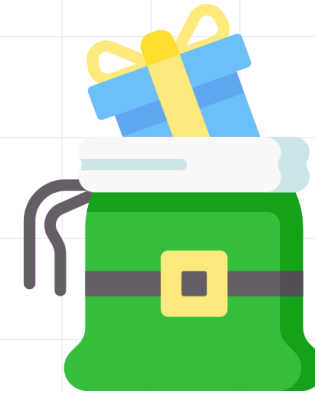
- Contents and number of elements can be changed
- Analogous to a sack

### ☐ Immutable

- Contents and number of elements cannot be changed
- Analogous to a crayon with its own position

### ☐ Necessity of immutable data type

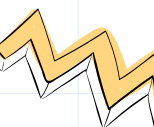
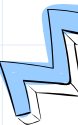
- Despite these inconvenient restrictions, immutable types are necessary because processing is simpler and faster than mutable data types.



mutable  
→ sack



immutable  
→ crayon  
box



# [5] Summary



■ Numeric data type: single/direct access/immutable

Numeric

**int**

- no decimal point
- -5, 0, 17

-5

**float**

- contains floating point
- 1.0, 0.005, 5763.23

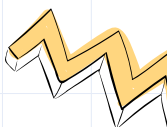
0.005

**complex**

- real part and an imaginary part
- -5+3j

-5

3



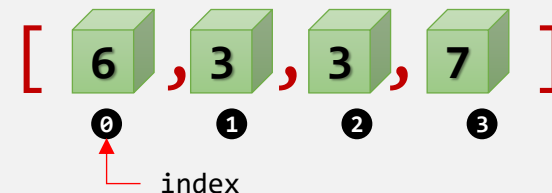
# [5] Summary



## Container

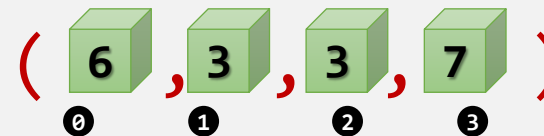
### List (list)

- Sequence access / mutable
- [-5, 1.7, ' abc ']



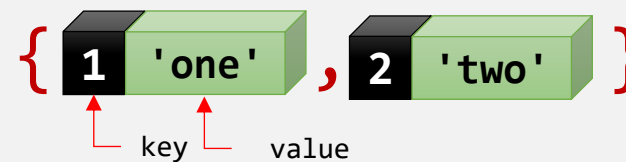
### Tuple (tuple)

- Sequence access / immutable
- (1,2,3)



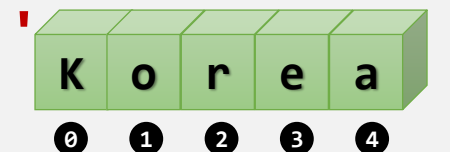
### Dictionary (dict)

- Mapping access / mutable
- {1:'one', 2:'two'}



### String ( str )

- Sequence access / immutable
- "Korea", ' Republic of Korea ', '2035'



### Set (set)

- Set access / mutable
- {1,2,3}

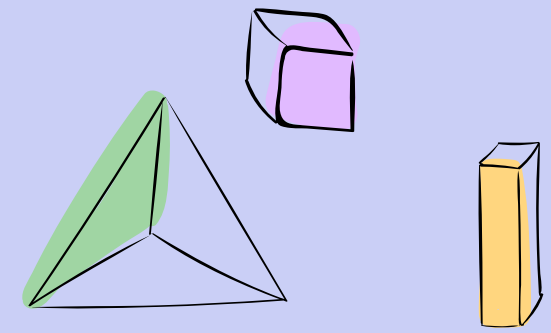
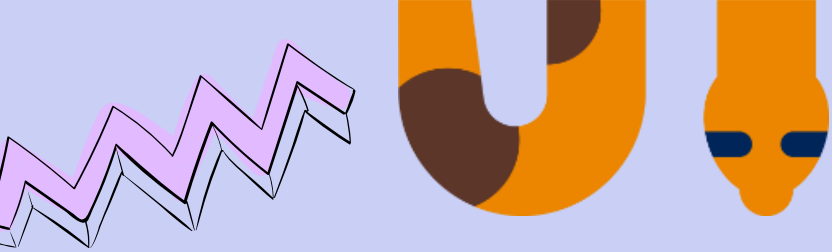




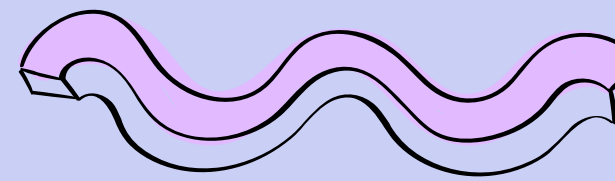
# [5] Summary

## Summary table of data types

| data type  | name  | number of elements | possibility of change | element access | characteristic                                                                                     |                                                                                                        |
|------------|-------|--------------------|-----------------------|----------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Numeric    | int   | single             | immutable             | direct         | <ul style="list-style-type: none"><li>• Order matters.</li><li>• Duplication not matters</li></ul> |                                                                                                        |
|            | float |                    |                       |                |                                                                                                    |                                                                                                        |
| complex    |       |                    |                       |                |                                                                                                    |                                                                                                        |
| String     | str   | container          |                       | sequence       |                                                                                                    |                                                                                                        |
| Tuple      | tuple |                    |                       |                |                                                                                                    |                                                                                                        |
| List       | list  |                    | mutable               |                | mapping                                                                                            | <ul style="list-style-type: none"><li>• Order not matters.</li><li>• Duplication not exists.</li></ul> |
| Dictionary | dict  |                    |                       |                |                                                                                                    |                                                                                                        |
| Set        | set   |                    |                       | set            |                                                                                                    |                                                                                                        |



# 02 list

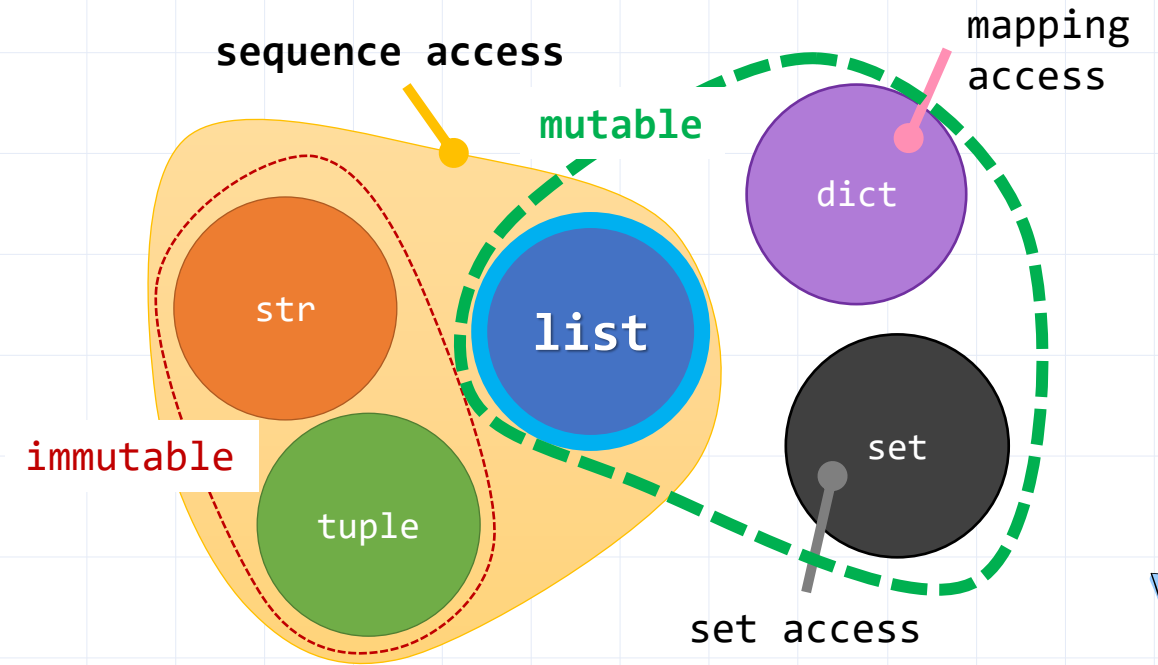


# [1] Characteristic of list

[1, 9, 3, 2]

## ■ Features

- **Container** data type
  - Store multiple data at once
- **Sequence** access(indexing)
  - The order of elements is important.
- **mutable** data type
  - The number and contents of elements can be changed at anytime.
- data type of elements
  - Any data type can be accepted as elements.



## [2] How to create

[1, 9, 3, 2]

■ **List**: A data type that can have multiple elements

□ How to create

○ **Direct creation**: List elements within square brackets `[]`, separated by commas `,`.

$L = [element_1, element_2, \dots, element_n]$

■ example

```
>>> L = [10, 'a', 'okey', 37.5]
>>> type(L)
<class 'list'>
```



**`type(variable)`** :

Built-in function to check data type

## [2] How to create

- Create with the conversion function *list* :

- Create a list by converting other data types with the conversion function.

```
L= list( other_data_type )
```

- Example: Converting a string to a list

```
>>> s = 'K-pop star'
>>> L = list ( s )
>>> L
['K', '-', 'p', 'o', 'p', ' ', 's', 't', 'a', 'r']
>>> type ( L )
< class 'list' >
```

## [2] How to create



### ○ Create by *comprehension*

- See Chapter ch.9
- Similar to mathematical expression
- Example: Use similar expressions to  $\{x \in N : x > 2\}$

```
>>> [ x for x in range ( 4 ) ]  
[ 0 , 1 , 2 , 3 ]  
>>> [ x ** 2 for x in range ( 10 ) if x % 2 == 1 ]  
[ 1 , 9 , 25 , 49 , 81 ]
```



# [3] Basic operations

## ■ Operators

- $+$  : concatenates two lists ( $a+b \neq b+a$ )

```
>>> a = [1, 2, 3]; b = [7, 8, 9]
>>> a + b
[1, 2, 3, 7, 8, 9]
```

- $*$   $n$  : repeats elements integer  $n$  times

```
>>> L = [1, 2, 3]
>>> L * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

# [4] Access to elements



L = [ , , ,  ]

①      ②      ③

## □ Accessing elements by *index*

- All sequence data types, list, string, and tuple, can use index
- to access elements.
- Index starts from 0
- Example

```
>>> L = [1, 9, 3, 2]
```

```
>>> L[1]
```

```
9
```

```
>>> L[1] + L[3]
```

```
11
```

## □ Changing elements

- List is mutable!!

```
>>> L[2] = 5
```

```
>>> L
```

```
[1, 9, 5, 2]
```

# [4] Access to elements



## □ Index of the last element

- The last element is numbered **-1**.
  - There is no need to count the order of the last element of a long list!!!
- Example)



```
>>> L =[1, 9, 3, 2]
```

```
>>> L[-1]
```

```
2
```

```
>>> L[-2]
```

```
3
```

```
>>> L[-4]
```

```
1
```

# [5] Slicing



□ *Slicing*: Extracting of some parts of list

○ Format

```
res= list_name[start=0 : stop=n : step=1]
```

- *res*: result of slicing
- *start*: cut from here
- *stop*: cut before this,
- *step*: interval of extraction
- *start*, *stop*, *step* can be omitted



TIP: from *start* before *stop* with *step* intervals!!!

# [5] Slicing

## ○ Example

```
>>> L = [1, 9, 3, 2]
>>> L[1:3:1]    # from 1 to before 3 by interval 1
[9, 3]
>>> L[2:3]      # from 2 to before 3
[3]
>>> L[0:4:2]    # from 0 to before 4 with 2 steps interval
[1, 3]
```

# [5] Slicing

## ○ *Partial replacement*

- the sliced part replaced by another list

L = [  ,  ,  ,  ]

①      ②      ③

```
>>> L =[1, 9, 3, 2]
```

```
>>> L [1:3]=[99, 101] # replace from 1 to before 3
```

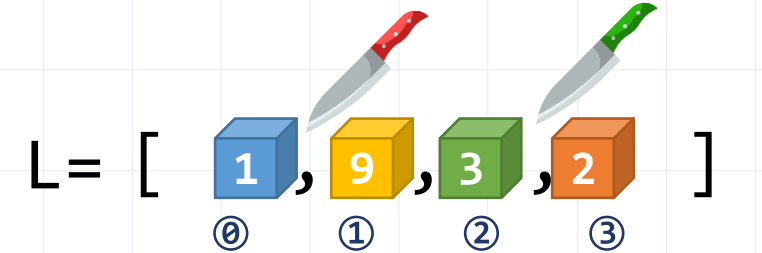
```
>>> L
```

```
[1, 99, 101, 2]
```

# [5] Slicing

## Partial deletion

- Delete the sliced part



```
>>> L =[1, 9, 3, 2]
>>> L [1:3]=[]    # delete from 1 to one before 3
>>> L
[1, 2]
```



Caution: Error if the step of the slicing is greater than 1

```
>>> L =[1, 9, 3, 2]
>>> L [::2]=[]
Traceback (most recent call last):
...
ValueError : attempt to assign sequence of size 0 to
extended slice of size 2
```

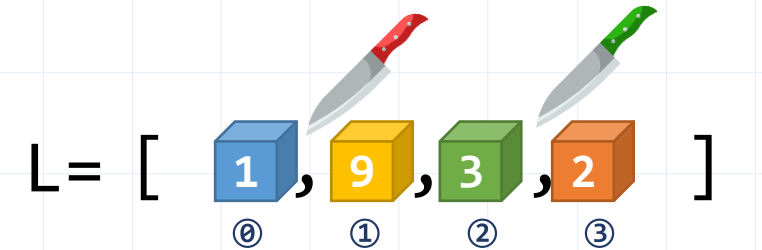
# [5] Slicing

## ○ *Omitting start, stop, and step*

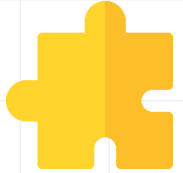
- Omitting start or stop
  - $L[:2]$  #  $L[0:2] \rightarrow [1, 9]$
  - $L[2:]$  #  $L[2:4] \rightarrow [3, 2]$
- Omitting all elements
  - $L[:]$  or  $L[::]$   $\rightarrow [1, 9, 3, 2]$
- Reversing elements
  - $L[::-1] \rightarrow [2, 3, 9, 1]$

## ○ *etc*

- empty list
    - $L[1:1] \rightarrow []$
    - $L[4:5] \rightarrow []$
- # cf. Error when accessing  $L[4]$



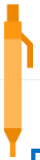
## [5] Slicing



Given two lists  $a=[1,4,7,9,10]$ ,  $b=[11,13,17,18,22]$  Extract the odd-numbered elements of  $a$  and the even-numbered elements of  $b$  from the two lists and place them in a new list  $c$ . Put the elements of  $a$  first, then the elements of  $b$  second.

```
>>> a =[1, 4, 7, 9, 10]; b =[11, 13, 17, 18, 22]
>>> c = a[::2]+b[1::2]
>>> c
[1, 7, 10, 13, 18]
```

# [5] Slicing



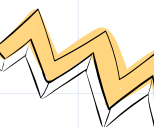
Predict the next outcome .

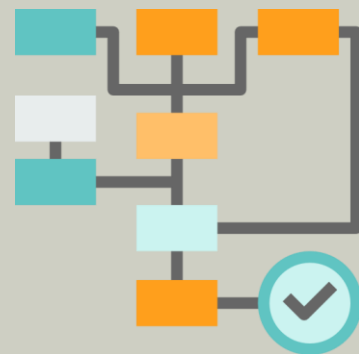
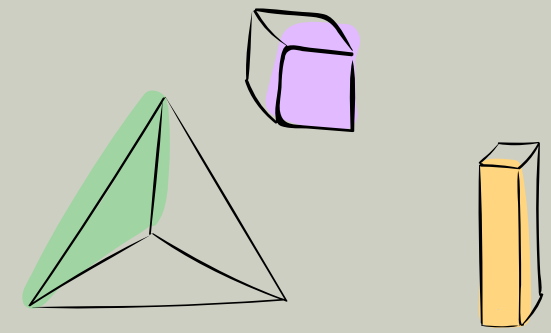
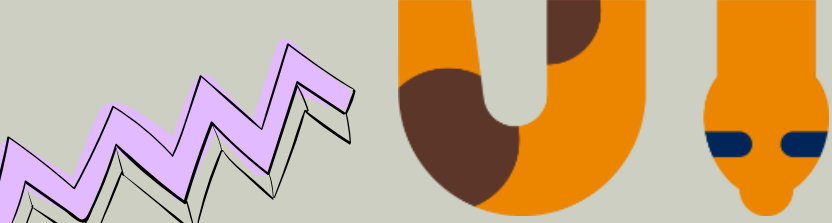
- ☐ `a[1:3]` → `[2, 3]`
- ☐ `a[3:5]` → `[4, 5]`
- ☐ `a[:]` → `[1 2 3 4 5]`
- ☐ `a[::]` → `[1 2 3 4 5]`
- ☐ `a[3:-1]` → `[4]`
- ☐ `a[::-1]` → `[5, 4, 3, 2, 1]`
- ☐ `a[2::-1]` → `[3, 2, 1]`
- ☐ `a[1:-2]` → `[2, 3]`
- ☐ `a[4:3]` → `[]`

`a = [1, 2, 3, 4, 5]`

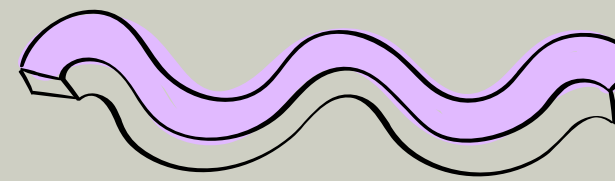
Indices: ① ② ③ ④  
Step: -2 -1

`[2, 3], [4, 5], [1, 2, 3, 4, 5], [1, 2, 3, 4, 5], [2, 3], [5, 4, 3, 2, 1], [3, 2, 1], [2, 3], []`





# 03 Methods of List



# [6] List method - Add element



## ■ Add element methodw

- *append(e)*: add an element *e* to the end

```
>>> L = [1, 9, 3, 2]; L.append(5)
>>> L
[1, 9, 3, 2, 5]
```

■ cf. `L=L+[5]` or `L+= [5]`

*L.append(e)*



- *extend(T)*: add a list *T* at the end

```
>>> L = [1, 9, 3, 2]; T = [6, 7]
>>> L.extend(T)
>>> L
[1, 9, 3, 2, 6, 7]
```

■ cf. `L=L+T` or `L+=T`

*L.extend(T)*

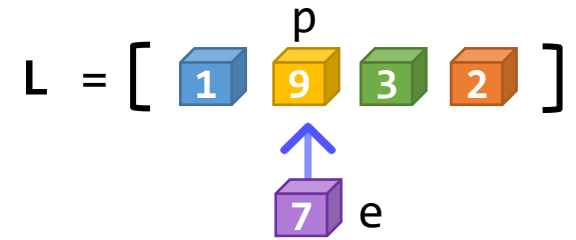


## [6] List method - Add element

□ *insert(p, e)*: insert element *e* at index *p*

```
>>> L =[1, 9, 3, 2]; L.insert (1, 7)
>>> L
[1, 7, 9, 3, 2]
```

*L.insert(p, e)*



## [6] List method - remove element

□ ***remove(e)***: remove element with value *e*

- If there are multiple, the first one is removed; if there are none, an error occurs.

```
>>> L =[1, 9, 3, 2]; L.remove(9) #remove 9
>>> L
[ 1 , 3 , 2 ]
```

***L.remove(e)***

L = [ 1 9 3 2 ]  
          e



If you remove an element that does not exist, an error will occur, so checking with the `in` operator is necessary.

## [6] List method - remove element



Delete all elements with zero from a list.

- Using the method `.remove(e)`

```
L =[1, 0, 0, 5, 0, 3, 0, 0, 2, 0, 1, 0]
while 0 in L :
    L.remove(0)
print(L)
```

```
[1, 5, 3, 2, 1]
```

## [6] List method - remove element

Write a program to remove all elements 4 and 7 from the list [1,3,4,4,3,2,7,4,1,7,4].

```
del_list = [4, 7]
L = [1, 3, 4, 4, 3, 2, 7, 4, 1, 7, 4]
for x in del_list :
    while x in L:
        s.remove(x)
print(L)
```

```
[1,3,3,2,1]
```

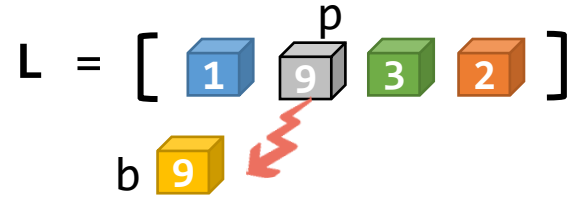
## [6] List method - remove element

### ■ Element removal method

□ `pop(p=-1)`: return an element at  $p$  and remove it.

```
>>> L = [1, 9, 3, 2]; b = L.pop(1)
>>> b;L
9
[1, 3, 2]
>>> L.pop() # pop the last
2
```

`b = L.pop(p)`



- The element is returned as it is removed from the list.
- If no position  $p$  is specified, it defaults to position -1, the last element.

## [6] List method - remove element

- *clear()*: Clear all elements

```
>>> L = [1, 9, 3, 2]; L.clear()
>>> L
[]
```

- same as `L = []`

*L.clear()*

`L = [ ]`

# [6] List method – reordering



## ■ reorder method

□ *reverse()*: Reverses the order.

```
>>> L = [1, 9, 3, 2]; L.reverse()  
>>> L  
[2, 3, 9, 1]
```

○ cf. `L=L[::-1]`

*L.reverse()*

L = [ 1 9 3 2 ]  
→ [ 2 3 9 1 ]

## [6] List method – reordering



□ `sort([reverse=False]):` sort

○ `L.sort()` ascending order

▪ [1, 2, 3, 9]

○ `L.sort(reverse=True)` descending order

▪ [9, 3, 2, 1]

cf. Equivalent to builtin function  
`sorted(L)`

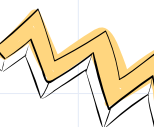
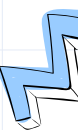
`L.sort()`

L = [ 1 9 3 2 ]

→ [ 1 2 3 9 ]

`L.sort(reverse=True)`

→ [ 9 3 2 1 ]



# [6] List method – reordering



## ○ example

```
>>> L = [1, 9, 3, 2]; L.sort() #Ascending order
```

```
>>> L
```

```
[1, 2, 3, 9]
```

```
>>> L = [1, 9, 3, 2]; L.sort(reverse=True) #Descending order
```

```
>>> L
```

```
[9, 3, 2, 1]
```

```
>>> L = [1, 9, 3, 2]; L=sorted(L) #Built-in function
```

```
>>> L
```

```
[ 1 , 2 , 3 , 9 ]
```

## [6] List method - example



Find a list excluding the top three elements from [-3.2, 5.5, 4.1, 1.1, -1.3, 2.7, 0.5]. However, the original order remains the same.

```
a = [-3.2, 5.5, 4.1, 1.1, -1.3, 2.7, 0.5]
temp = a.copy() # copy cf. temp=a
temp.sort(reverse = True) # Sort in descending order
print(f'sorted = { temp }')
for x in temp [: 3 ]:
    while x in a :
        a.remove(x)
print(f'removed = { a }')
```

```
sorted=[5.5, 4.1, 2.7, 1.1, 0.5, -1.3, -3.2]
removed=[-3.2, 1.1, -1.3, 0.5]
```

## [6] List method



### Copy of mutable data

- Even if you execute `b=a` when there are lists `a` and `b`, the two lists will have the same address, so you need to be careful.

```
>>> a =[ 1 , 2 , 3 ]; b = a
>>> b [ 0 ] = 100
>>> a
[ 100 , 2 , 3 ]
>>> id ( a ); id ( b )
3134370962248
3134370962248
```

## [6] List method

- List copy method or slicing Using ***b=a[:]*** creates a new object.

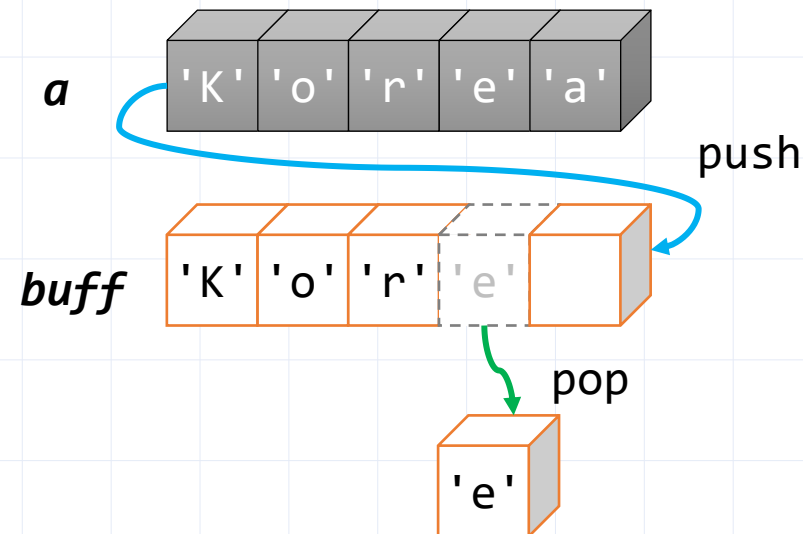
```
>>> a = [1, 2, 3]; b = a.copy()
>>> b[0] = 100
>>> a
[1, 2, 3]
>>> id(a); id(b)
3134370962568
3134370962376
```

## [6] List method - example



### LIFO (Last In First Out) example

- There is a list `a=['K','o','r','e','a']`. Pick four items, one from the front, and add them to the empty list `buff` in order from front to back. When removing from `buff`, pick only three items from the back. When adding to `buff`, use the `append` method. When removing from `buff`, use `pop`.



# [6] List method - example



code

```
a = ['K', 'o', 'r', 'e', 'a']  
buff = []  
for e in a[:4]:  
    buff.append(e)  
for _ in range(3):  
    b = buff.pop()  
    print(b)
```

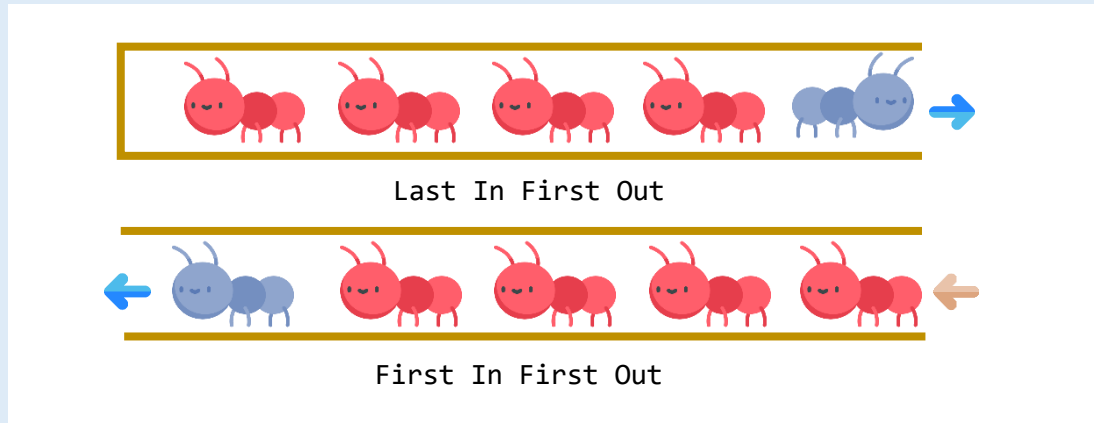
e  
r  
o

# [6] List method



What is *LIFO*(Last In First Out)?

- the data that comes in latest always comes out first – *stack*.



○ cf. First In First Out (*FIFO*) – *queue*

# [6] List method



## ■ Element search methods

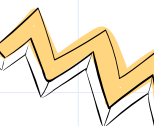
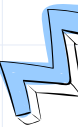
$L = [1, 9, 3, 2]$

- ***count(e)***: Count the number of elements with value  $e$

```
>>> L = [1, 9, 3, 4, 3, 2]
>>> L.count(3)    # Number of 3
2
```

- ***index(e[,p=0])***: search for the position of element  $e$  from index  $p$ .
  - If you omit  $p$ , it starts from 0

```
>>> L = [1, 9, 3, 4, 3, 2]
>>> L.index(3)    # From 0
2
>>> L.index(3, 3)    # From 3
4
```



## [6] List method - examples

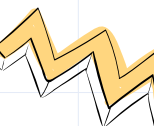
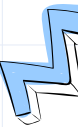


Leave only four zeros(0) from a list.

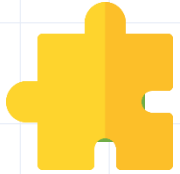
- For a list `[1,0,0,5,0,3,0,0,2,0,1,0]`, delete the 0 elements from the list while leaving four 0.
- Use *count* method.

```
s =[1, 0, 0, 5, 0, 3, 0, 0, 2, 0, 1, 0]
while s.count(0) > 4:
    s.remove(0)
print(s)
```

```
[1, 5, 3, 0, 0, 2, 0, 1, 0]
```



## [6] List method - examples



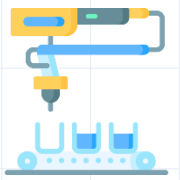
Find the locations of the same element

- For a list `[7,8,8,7,5,6,8,9,8]`, find all indices for element 8 and print them .

```
s = [7, 8, 8, 7, 5, 6, 8, 9, 8]
n = s.count(8)
L = []; p = 0
for _ in range(n):
    p = s.index(8, p)
    L.append(p)
    p += 1
print(L)
```

```
[1, 2, 6, 8]
```

# [7] Useful built-in functions



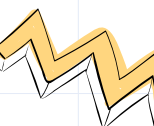
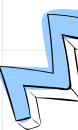
- Important *built-in* functions applicable to container data types including list

- `len(o)`: Find the number of elements of object `o`

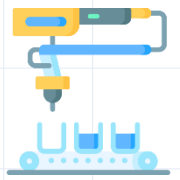
```
>>> L = [1, 7, 8, 4, 6]; len(L)
5
```

- `sorted(o)`: Sorts the elements of `o`

```
>>> L = [1, 7, 8, 4, 6]; sorted(L) # ascending order
[1, 4, 6, 7, 8]
>>> sorted(L, reverse = True) # descending order
[8, 7, 6, 4, 1]
>>> print(L.sort())           # method
[1, 4, 6, 7, 8]
```



# [7] Useful built-in functions



## ■ *del* statement

- *del o*: It is possible to delete the entire object *o*, and if expressed in slicing format, specified elements can be deleted.

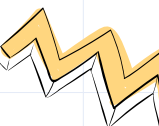
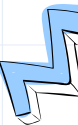
```
>>> L = [1, 7, 8, 4, 6]; del L[:3]
>>> L
[4, 6]
```



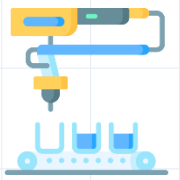
## Caution on using *sorted*

- It is not possible to use it on lists consisting of elements that cannot be compared to each other.

```
>>> L = [1, 2, 'a', 'z', 3]; sorted(L)
Traceback ( most recent call last ):
...
TypeError : '<' not supported between instances of 'str' and 'int'
```



## [7] Useful built-in functions



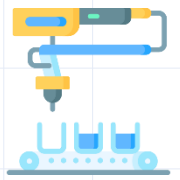
 In the list `[1,7,8,4,6]`, delete all elements in odd-numbered positions.

```
>>> L =[1, 7, 8, 4, 6]; del L[::2]
```

```
>>> L
```

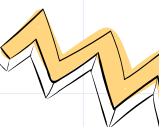
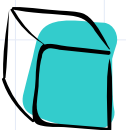
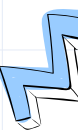
```
[7, 4]
```

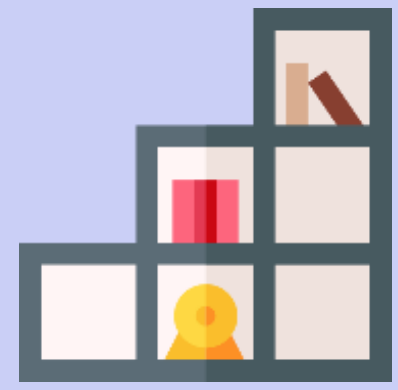
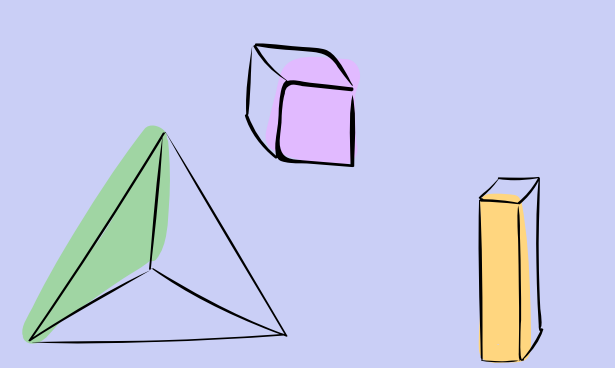
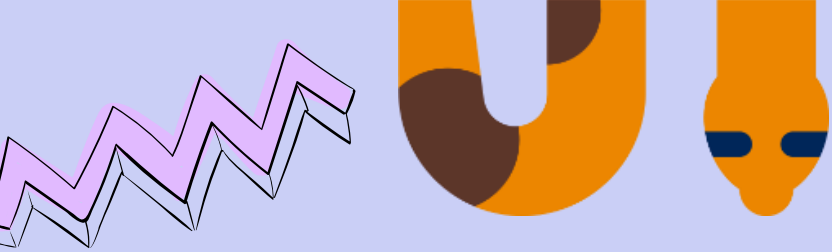
# [7] Useful built-in functions



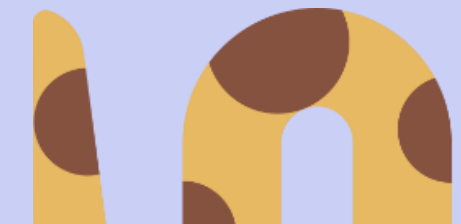
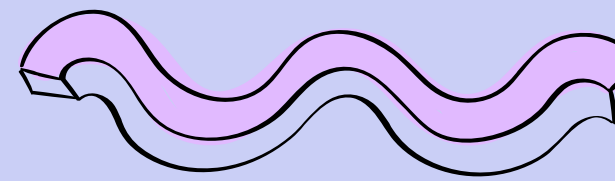
Difference between *clear* method and *del* statement

- ☐ For the list *a*
- ☐ *a.clear()*
  - The elements in *a* are deleted, resulting in an empty list [] .
- ☐ *del a*
  - object *a* in memory is eliminated.





# 04 Tuple



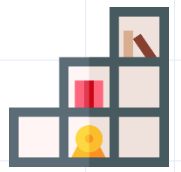
# [1] Features of tuple

(, , , )

■ **Tuple:** A data type that can have multiple elements

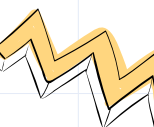
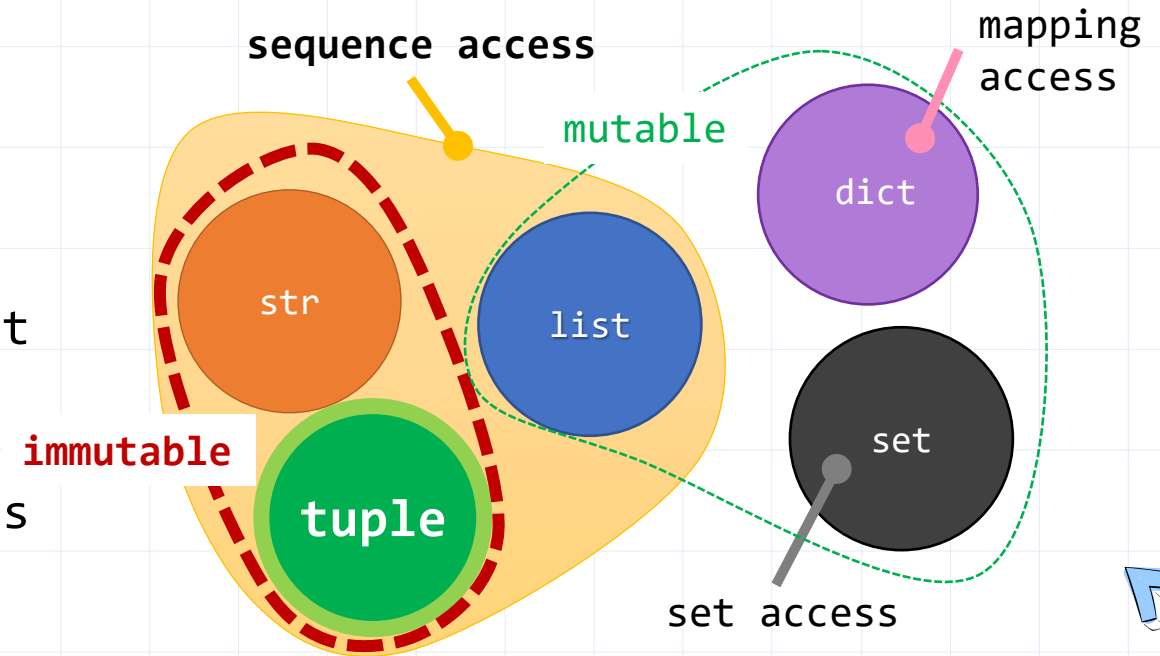
- Elements are separate by commas 『,』 in round bracket『()』.
- Because it is immutable, the processing is simpler and faster than a list.
- Tuple is commonly used when
  - the position of an element has meaning.
  - the number of items does not change.
- Examples:
  - spatial coordinates, color, date, time
    - (x, y, z), (r, g, b), (yyyy, mm, dd), (hour, min, sec)

# [1] Characteristics (1, 9, 3, 2)

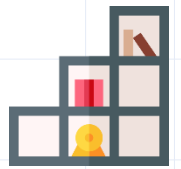


## ■ Key Features

- **Container** data type
  - Store multiple elements at once
- **Sequence** access (indexing)
  - The order of elements is important
- **immutable** data type
  - The number and content of elements cannot be changed.
- data type of elements
  - can be of any data type



## [2] How to create



(, , ,  )

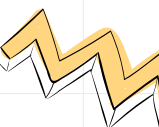
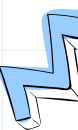
### ■ How to create tuple

- Direct creation: Elements are separated by commas『,』 within round bracket『()』.

```
tuple_name = (element1, element2,... elementn)
```

○ example

```
>>> t =(1, 2, 3)
>>> type(t)
<class 'tuple'>
```



## [2] How to create

- Created with the conversion function *tuple*

```
tuple_name = tuple(other_data type)
```

- Example: Convert a string and a list to a tuple

```
>>> tuple('abcd')      # from string
('a', 'b', 'c', 'd')
>>> tuple([1, 2, 3])   # From the list
(1, 2, 3)
```

## [2] How to create

### □ Special creation example

#### ○ Empty tuple

```
>>> t = ()      # empty tuple
```

#### ○ Single element tuple

```
>>> t = (1,)    # tuple of 1 element  cf. t=(1)
```

type(t)??

#### ○ Short form

##### ■ Omit parentheses

```
>>> t = 1, 2, 3  # Packing to tuple
>>> type(t)
<class 'tuple'>
>>> a, b, c = 1, 2, 3 # Unpacking of tuples
>>> a
1
```



## [3] Operations of tuple

□  $+$  : concatenate two tuples

```
>>> t = (1,) + (1, 3)
>>> t
(1, 1, 3)
```

□  $*$   $n$  : repeat all elements *integer*  $n$  times

```
>>> t = 3*(1,)
>>> t
(1, 1, 1)
```

# [4] Accessing Elements



t = ( , , ,  )

①      ②      ③

## □ Indexing: Access elements by index

- List, string, and tuple, which are sequential data types, are all the same for accessing
- Index starts from 0
- example

```
>>> t = (1, 9, 3, 2); t[1]
9
>>> t[1] + t[3]
11
```

## □ Elements cannot be changed

- *immutable* data type

```
>>> t = (1, 9, 3, 2); t[2] = 33
Traceback (most recent call last):
....
t[2]=33
TypeError : 'tuple' object does not support item
assignment
```

# [4] Accessing Elements



t = ( , , ,  )

①      ②      ③

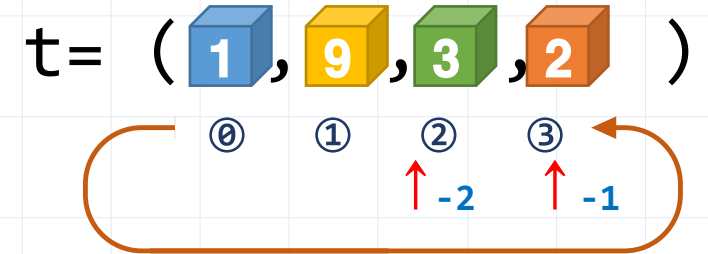
- Tuple creation and identification:
  - It is possible to create a new one

```
>>> t = (1, 9, 3, 2); id(t)
3081939686248
>>> t = (t[0], t[3], t[2]) # new definition
>>> t; id(t)
(1, 2, 3)
3081939603104
```

## [4] Accessing Elements

### □ Last Element: 『-』indexing

- The index of the last element is numbered **-1**.
- example



```
>>> t = (1, 9, 3, 2)
>>> t[-1]; t[-2]; t[-4]
2
3
1
```

# [5] Slicing

□ *Slicing*: Extracting of some parts of tuple

○ Format

```
res= tuple_name[start=0 : stop=n : step=1]
```

- *res*: result of slicing
- *start*: cut from here
- *stop*: cut before this,
- *step*: interval of extraction
- *start*, *stop*, *step* can be omitted

t= (  ,  ,  ,  )

①   ②   ③

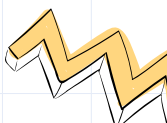
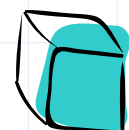
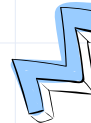


TIP: from *start* before *stop* with *step* intervals!!!

# [5] Slicing

## ○ Example

```
>>> t = (1, 9, 3, 2); t[1:3]
(9, 3)
>>> t[1:4:1]
(9, 3, 2)
>>> t[0:5:2]
(1, 3)
```



# [6] Method



## ■ Search method

- `count(e)`: count the number of elements `e` in tuple.

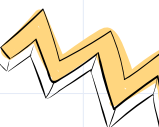
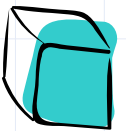
```
>>> t = (1, 7, 7, 8, 6, 7, 11, 8, 5); t.count(7)
3
```

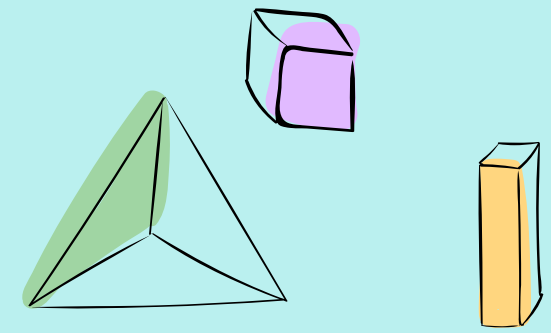
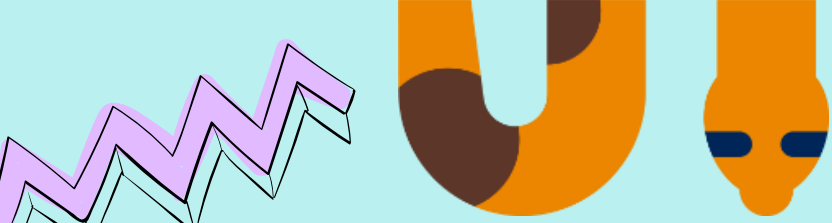
- `index(e[, p=0])`: search for the position of element `e` from `p`.

```
>>> t = (1, 7, 7, 8, 6, 7, 11, 8, 5)
>>> t.index(7) # From position 0
1
>>> t.index(7, 3) # From position 3
5
```



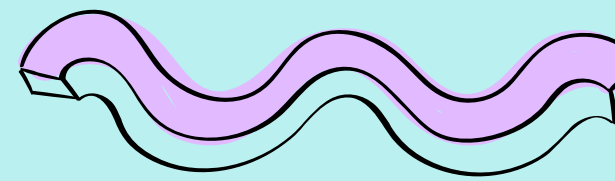
Tuple is *immutable*, so there are fewer methods, but processing speed is faster than list.





"a5  
가 & !

## 05 String

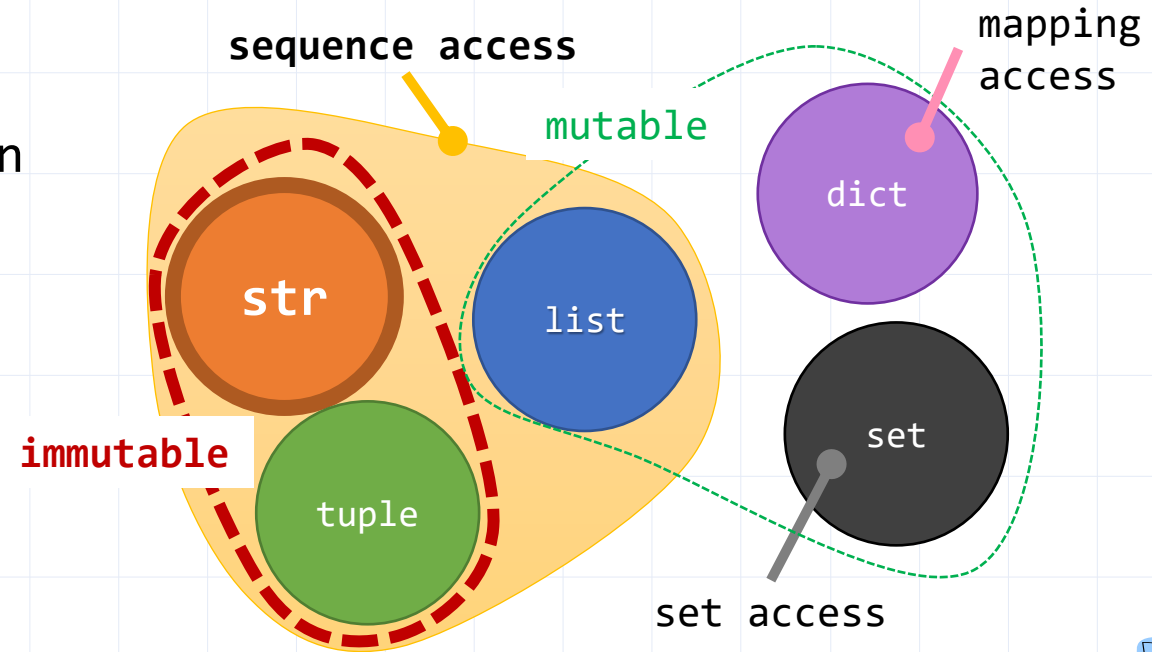


# [1] Features

"a5  
가&!"

## ■ Features

- String : One-dimensional collection of multiple characters
- Store multiple data at once
  - Container
- Elements are characters
- The order of elements is important
  - Sequence access
- The number and content of elements cannot be changed.
  - immutable



## [2] How to create



"Apple"

### ■ How to create

#### □ Direct creation :

○ Creating a string in a line using `' '` or `" "`

■ example

`s = 'C1C2 ... Cn'` or `"C1C2 ... Cn"`

```
>>> s = "I'm a boy."          # '
>>> s = 'He said "No!".'      # "
>>> s = 'Tom\'s mom shouted "He isn\'t here."' # ' and "
```

## [2] How to create

- Multi-line strings (including new line): Use triple quote `'''` or `"""`
  - Example

```
>>> s = '''name: alex
student ID: 12345'''
>>> s
'name: alex\nstudent ID: 12345'
>>> print(s)
name : alex
student ID : 12345
```

## [2] How to create

- Create a string with the conversion function *str*

```
s = str(other_data type)
```

- Example : Convert real number to string

```
>>> f = 12.3567  
>>> str(f) # to string  
'12.3567'
```

# [3] Basic operations



- `+` : concatenate two strings

```
>>> 'lady' + 'bug'
'ladybug'
```

- [Note] In the case of nearby string constants, they are concatenated

```
>>> 'Snow' 'board'
'Snowboard'
```

- `*` *n* : repeat elements of the string *n* times

```
>>> s = '[o_o]'
>>> 5 * s
'[o_o][o_o][o_o][o_o][o_o]'
```

# [3] Basic operations



## *Immutable* data type

□ Modification is not possible, but it is possible to create a new one with the same name.

□ Example

```
>>> p = 'Blue' ; q = 'Frog'
>>> id(p)
1780554306480
>>> p = p + q # create new
>>> id(p)
1780545198576
>>> p = p * 3 # Create a new one
>>> id(p)
1780554363504
>>> print(p)
BlueFrogBlueFrogBlueFrog
```

# [4] Accessing Elements



S = "A p p l e"

① ② ③ ④

## □ Accessing elements using an index

- List, string, and tuple, which are sequence data types, are all the same for accessing
- Index starts from 0
- Example

```
>>> s = "Apple"; s[1]
'p'
>>> s[3] + s[4]
'le'
```

## □ Element cannot be changed

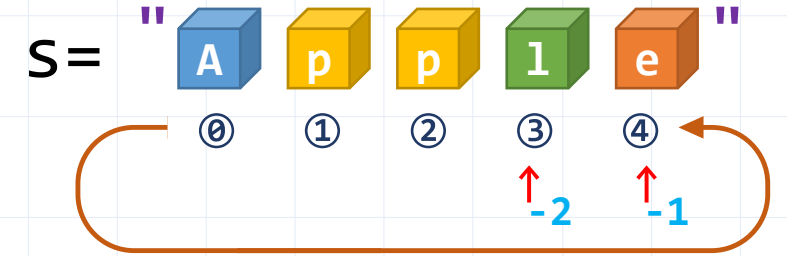
- immutable

```
>>> s = "Apple"; s[2] = 'l'
Traceback (most recent call last):
...
TypeError : ' str ' object does not support item
assignment
```

# [4] Accessing Elements

## □ Last Element: 『-』indexing

- Like a list, the last element is numbered **-1**.
- Example
  - `s[-1] → 'e'`
  - `s[-2] → 'l'`
  - `s[-4] → 'p'`



# [5] Slicing

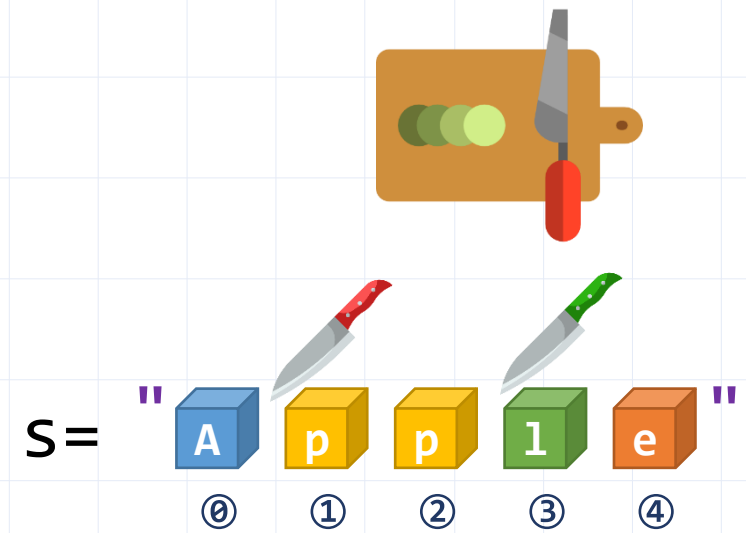
- Slicing: extracting some elements of a string

○ Form

```
res= string_name [start=0 : stop=n : step=1]
```

- *res*: result, *start*: start, *stop*: before it, *step*: interval
- *start*, *stop* and *step* can be omitted

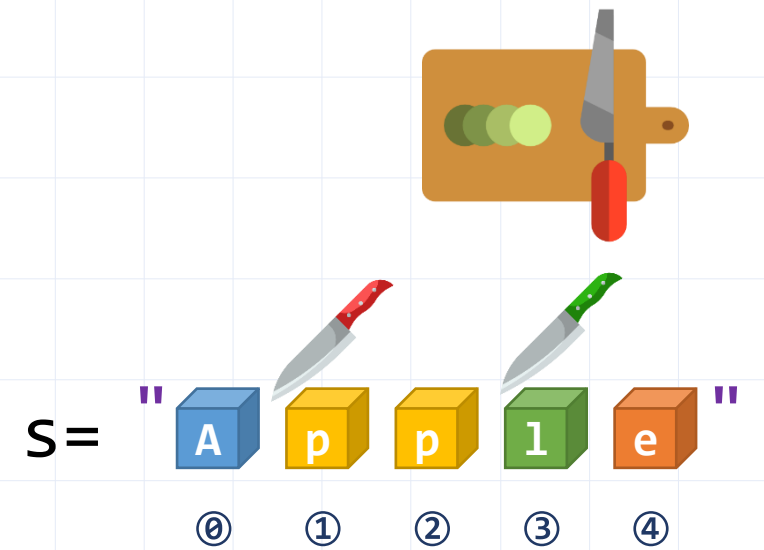
TIP: from *start* before *stop* with *step* intervals!



# [5] Slicing

## ○ Example

```
>>> s = "Apple"; s[1:4]
'ppl'
>>> s[2:4:1]
'pl'
>>> s[0:5:2]
'Ape'
```



# [5] Slicing

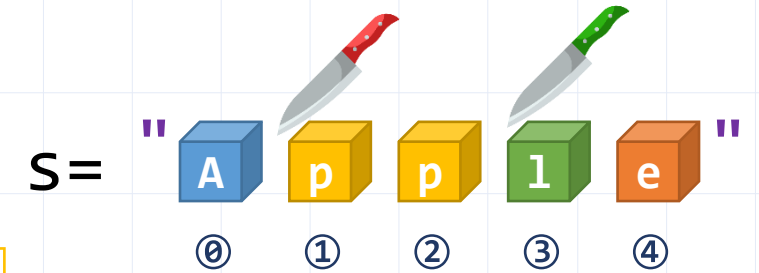
- Omitting arguments

- *skip start*

```
>>> s = "Apple"; s[:2]
'Ap'
>>> s[2:]
'ple'
```

- *empty string*

```
>>> s="Apple"; s[1:1]
''
>>> s[6:8] # not exist
''
```



# [5] Slicing

- *all elements*

```
>>> s = "Apple"
>>> s[:]; s[::]
'Apple'
'Apple'
```

- *In reverse order*

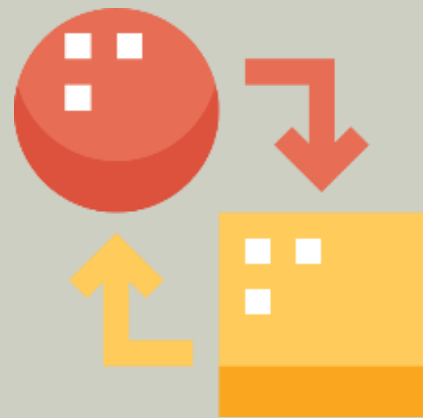
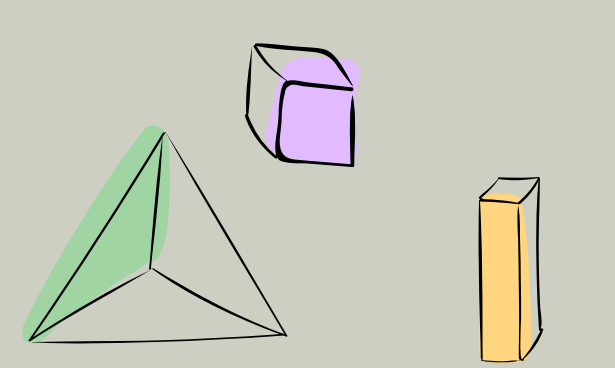
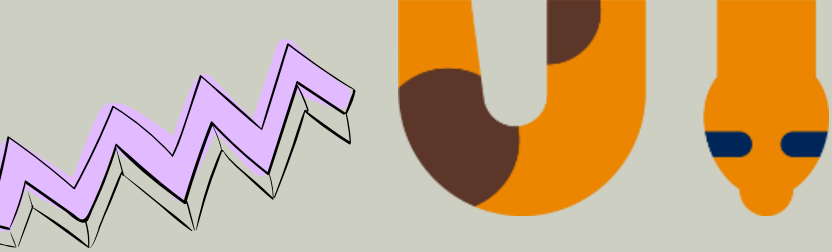
```
>>> s = "Apple"; s[::-1]
'elppA'
```



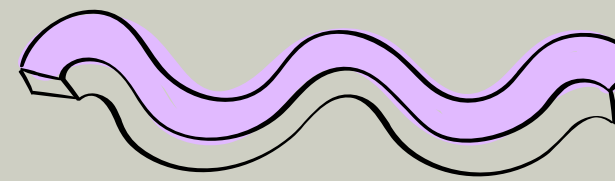


- Predict the next outcome.

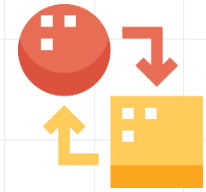
- s = 'crazy dog'**



# 06 Method of String



# [1] String method



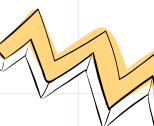
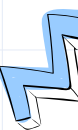
## ■ Main conversion methods

### □ *split* method

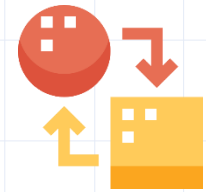
```
res = s.split([c=' '])
```

- Finds the string *c* in the string *s* and returns a list *res* whose elements are all string separated using delimiter *c*.
- If the argument *c* is omitted, it is assumed to be whitespace characters as a default.
- Example

```
>>> s = '12.3#3.65'; s.split('#')  
['12.3', '3.65']
```



# [1] String method



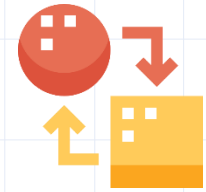
## □ join Method :

```
res = s.join(q)
```

- The join method concatenates all the elements of *q* by inserting delimiter *s* between all elements, generate a new string *res*.
- The *join* method is a kind of inverse operation of *split* method
- Example

```
>>> L = ['12.3', '3.65']  
>>> '#'.join(L)  
'12.3#3.65'  
>>> L = ['1', '2', '3', '4']; '->'.join(L)  
'1->2->3->4'  
>>> '0'.join('|||||')  
'|0|0|0|0|'
```

# [1] String method



## □ *upper* method

```
res = s.upper()
```

- Returns string *res* with all characters in the string *s* capitalized.

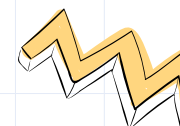
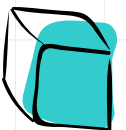
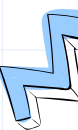
```
>>> s = 'ice cream cake'; s.upper()  
'ICE CREAM CAKE'
```

## □ *Lower* method

```
res = s.lower()
```

- Returns a string with all characters in the string *s* converted to lowercase.

```
>>> s = 'ICE CREAM CAKE'; s.lower()  
'ice cream cake'
```



# [1] String method

## □ *replace* method

```
res = s.replace(s1, s2)
```

- Returns a new string *res* in which all strings *s1* in string *s* are replaced with string *s2*.

```
>>> 'riding, voting'.replace('ing', 'es')  
'rides, votes'
```

# [1] String method

- *strip* method:

```
res = s.strip()
```

- Returns a string with all blank space characters (' ', '\t', '\n') before and after the string *s* deleted.

```
>>> 'A glass of cold beer\t\ n'.strip()  
'A glass of cold beer'
```

# [1] String method

## ■ String Inspection Methods

### □ *count* method

```
res = s.count(c)
```

- From string *s*, search for string *c* and returns the number of *c*.
- Example

```
>>> s = 'Big frog, small frog, king frog'
>>> s.count('frog')
3
```

# [1] String method

## □ *find* method

```
res = s.find(c[,p=0])
```

- From string *s*, find and return the position of string *c* starting from index *p*.
- Example

```
>>> s = 'Big frog, small frog, king frog'
```

```
>>> s.find('frog' , 4)
```

```
4
```

```
>>> s.find('frog', 5)
```

```
16
```

# [1] String method

## □ *startswith* method

```
res = s.startswith(c)
```

- Checks if the string *s* begins with the string *c* and returns the result as True or False.

```
>>> s = 'Big frog, small frog, king frog'
>>> s.startswith('Big')
True
```

- Example

```
>>> import math
>>> [s for s in dir(math) if s.startswith('a')]
['acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2',
'atanh']
```

# [1] String method

## □ *endswith* method

```
res = s.endswith(c)
```

- Checks whether the string *s* ends with *c*.

```
>>> s = 'Big frog, small frog, king frog'
>>> s.endswith('Frog')
False
```

## □ Other inspection methods

- *'isalpha'*, *'isascii'*, *'isdecimal'*, *'isdigit'*, *'isidentifier'*, *'islower'*, *'isnumeric'*, *'isprintable'*, *'isspace'*, *'istitle'*, *'isupper'*



# Summary

## ■ Number of elements:

- ☐ *Single*: one element - numeric/logical,
- ☐ *Container*: multiple elements - list/tuple/string/dictionary/set, etc.

## ■ Element Access method:

- ☐ *Direct* access: direct value substitution - numerical/logical,
- ☐ *Sequence* access: access using an index - list/tuple/string,
- ☐ *Mapping* access: access by key - dictionary,
- ☐ *Set* access: no direct element access - set

## ■ Element modifiability:

- ☐ *mutable*: elements and number of elements can be changed - list/dictionary/aggregation,
- ☐ *immutable*: elements and the number of elements cannot be changed.: numeric/logical/tuple/string



# Summary

## ■ List: enclose in '[]' , sequence, mutable data type

- How to create: create directly, convert with list function, and use list comprehension.
- Elemental access: using indexes for '[]'
- Slicing: ListName[start=0: stop=n: step=1]

## ■ Methods of List

- Adding element methods: **append**(e) -add e to end, **extend**(T) -append list T to end, **insert**(p,e) -insert e before position p
- Removing element methods: **remove**(e) -remove element e, **pop**(p=-1) -remove element at position p And return
- Reordering element methods: **reverse**() -reverse order, **sort**(reverse=False) -sort ascending order
- Searching element methods: **count**(e) -count the number of element e, **index**(e[,p=0]) -return the index of element e



# Summary

## ■ Tuple: enclose in `()`, **sequence** and **immutable** data type.

- How to create: create directly, convert with tuple function
- Elemental access: Use indexes on `[]`, slicing
- Searching element methods: **`count(e)`** -count the number of element e, **`index(e[,p=0])`** -return the index of element e

## ■ String: enclose in `'` or `"`, **sequence**, **immutable** data type.

- Creating multi-line strings: Use `'''` or `"""`.
- Elemental access: Use indexes on `[]`, slicing
- Transform methods: **`s.split(c='')`** - returns a list separated by c, **`s.join(L)`** -joins elements of L with s, **`s.replace(s1,s2)`** - replaces string s1 in s with s2, other methods - **`upper()`**, **`lower()`**, **`strip()`**
- Inspection methods: **`count(c)`** -count the number of times the string c occurs, **`find(c[,p=0])`** -find the start of the string c, **`startswith(c)`** -check if it starts with the string c, other methods - **`isalpha()`**, **`isdigit()`**, **`isupper()`**, **`islower()`**

